Data structure and Programming II

TP13: Graph

- Objective:
  - To practice implementing graph data structure in program from scratch
  - Apply C++ programming
- Individual work
- Submit to Moodle (do not zip)
  - *A pdf report (cover, exercise and solution with screenshot)*
  - *Source codes*

Deadline:
6 days

---

1. Write a C++ program that implements an undirected graph using an adjacency matrix.

```
void initialize(){
    // code
}
void addEdge(int u, int v, int weight = 1){
    // code
}
bool hasEdge(int u, int v){
    // code
}
void display(){  // code }
```

```
Adjacency Matrix:
   0 1 2 3 4
0: 0 1 1 0 0
1: 1 0 0 1 0
2: 1 0 0 0 1
3: 0 1 0 0 1
4: 0 0 1 1 0

Edge (1,3) exits: Yes
Edge (2,4) exits: Yes
Edge (0,3) exits: No
```

Example:

```
addEdge(0, 1);
addEdge(0, 2);
addEdge(1, 3);
addEdge(2, 4);
addEdge(3, 4);
```

2. Write a C++ program that implements an directed and weighted graph using an adjacency matrix.

```
void initialize(){
    // code
}
void addEdge(int u, int v, int weight){
    // code
}
bool hasEdge(int u, int v){
    // code
}
void display(){  // code }
```

Example:

```
addEdge(0, 1, 3);
addEdge(0, 2, 4);
addEdge(1, 3, 5);
addEdge(2, 4, 2);
addEdge(3, 4, 3);
addEdge(4, 2, 2);
```

```
Adjacency Matrix:
   A B C D E
A: 0 3 4 0 0
B: 0 0 0 5 0
C: 0 0 0 0 2
D: 0 0 0 0 3
E: 0 0 2 0 0

Edge (1,3) exits: Yes
Edge (2,4) exits: Yes
Edge (0,3) exits: No
```

3. Write a C++ program that implements an undirected graph using an adjacency list. Take the use of array of Linked List.

```
struct Node{
    // code
};
struct List{
    // code
};
 List* createList(){
    // code
}
void add_end(List* ls, int data){
    // code
```

```
Adjacency List

0 -> 1 2 3

1 -> 0 4

2 -> 0 5

3 -> 0 6

4 -> 1 7

5 -> 2 7

6 -> 3 7

7 -> 4 5 6
```

```
}
void displayList(List* ls){
    // code
}
List** createGraph(int vertices){
    // code
}
void addEdge(List** graph, int u, int v){
    // code
}
void displayGraph(List** graph, int vertices){
    // code
}
```

Example:

```
addEdge(graph,0,1);
addEdge(graph,0,2);
addEdge(graph,0,3);
addEdge(graph,1,4);
addEdge(graph,2,5);
addEdge(graph,3,6);
addEdge(graph,4,7);
addEdge(graph,5,7);
addEdge(graph,6,7);
```